可怕的浮点数

画布对图像的描述是通过数组记录每个**完整像素点**的色值数据，这也意味着无法直接并准确记录一个非完整像素点的色值，如 `0.5` 个像素点。

1）如何处理浮点数像素值

既然杯子里不能一边有水一边没水（色值），那就让它流满杯子，但水的高度（透明度）也会随之下降。

这里的流满就是将非完整的像素点的色值直接填充所处的完整像素格，但因为色值的扩散减弱也导致色值的透明度下降。

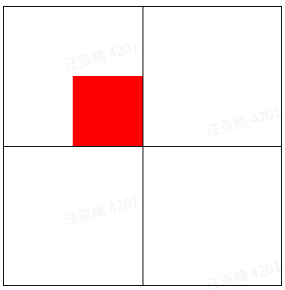
> 这里说的浮点数像素问题，专业名词是：亚像素问题，这个要深入挺复杂的，但无论是使用什么算法归根到底都是要算出这个“铺满像素格”后的色值，按照我上面说的便于理解。如果要深入可以看看官方解释。

然而色值铺满完整像素虽能解决浮点数像素值问题，但却加剧另一个问题：锯齿问题。

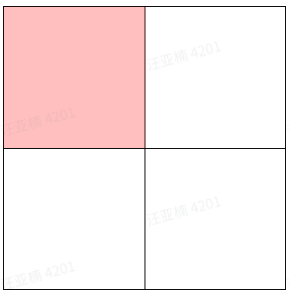
这还没完，画布赠送的边缘平滑服务会让加剧的锯齿变成了一个新的问题：边缘模糊。

接下来我们来看看下面的例子：

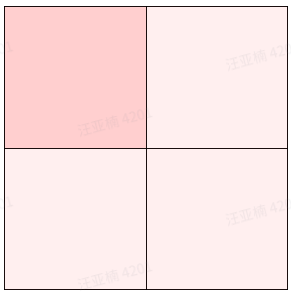
```
const canvas = document.createElement('canvas');
canvas.width = 2;
canvas.height = 2;
const ctx = canvas.getContext('2d');

ctx.fillStyle = 'rgb(255, 0, 0)';
ctx.fillRect(0.5, 0.5, 0.5, 0.5);
```



①理想的效果



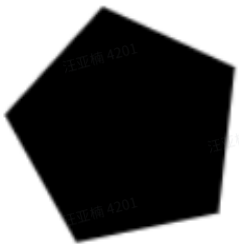
②铺满效果

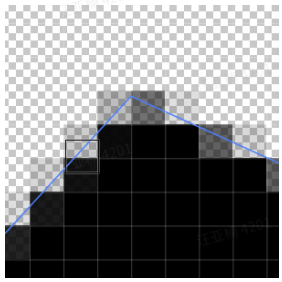


③平滑后效果

2）为什么会加剧锯齿问题？

屏幕中的每个像素点都是方形的，图像正是由这些方形格子组成，当我们放大图像看：



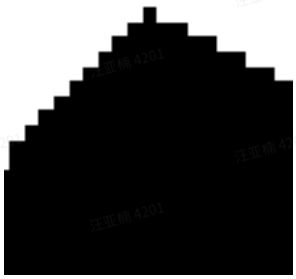


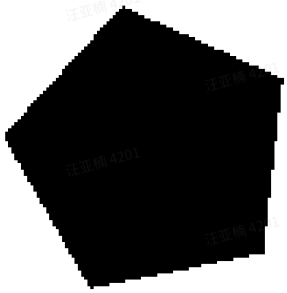
像素的方形会更加明显，这些“阶梯状毛刺”会使得边缘并非如我们想象中的平滑，也就是我们所说的“锯齿”。

可以想象，当这些方形格子越大，锯齿也就越大；这也就是为什么当浮点数像素铺满整个像素格后会加剧锯齿问题。

3）边缘平滑（抗锯齿）

边缘平滑是为了柔化上面所说的“毛刺”，上图已经经过了平滑处理，如果不加平滑处理，就会像下图一样，图像整体看起来边界锯齿会十分明显：



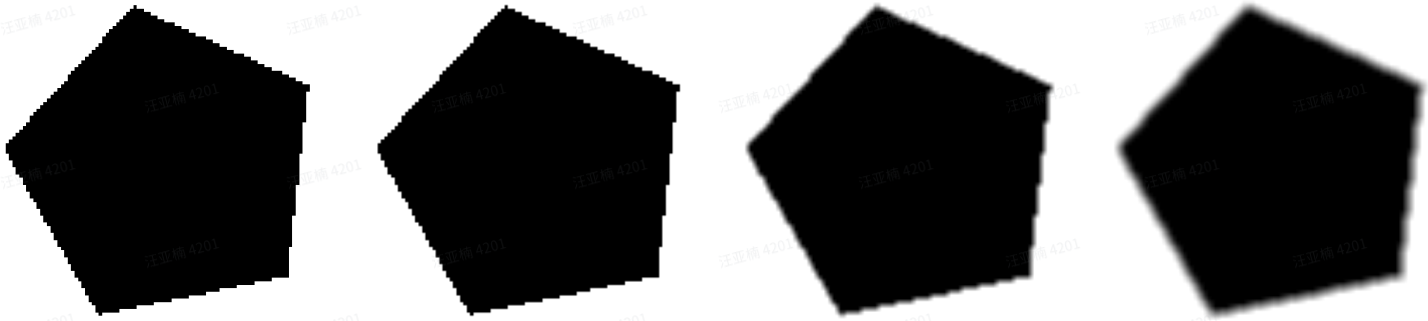


现在能实现边缘平滑的有很多种算法，但是具体画布使用了哪种我并没有找到相关文献。

> wiki中说：（Multisample anti-aliasing）Most modern GPUs are capable of this form of anti-aliasing。
>
> 所以我们先假定画布就是用的**多重采样抗锯齿**，这里大概讲讲，想稍稍深入可以先看这篇文章：抗锯齿 - 掘金

多重采样抗锯齿算法的原理就是在取区域像素所有色块的均值作为中心像素的色值，拿上面五角星做示例：

https://codepen.io/huanjinliu/pen/PoxeVdo?editors=1010

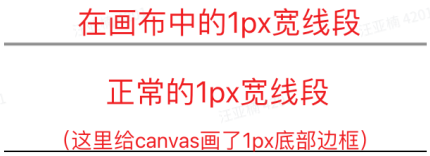


原图　　9点采样　　121点采样　　441点采样

可以看到当采样点越多的情况下，边缘看起来也更加平滑，但同时也会更模糊，对于PNG图像而言，图像边缘外的透明像素在平均色值后会导致区域出现大量半透明度像素，在我看来这也就是导致边缘模糊最直接原因。

4）浮点数引发的模糊描边惨案

```
const canvas = document.getElementById('canvas');
const ctx = canvas.getContext('2d');

ctx.moveTo(50, 75);
ctx.lineTo(250, 75);

ctx.lineWidth = 1;
ctx.stroke();
```

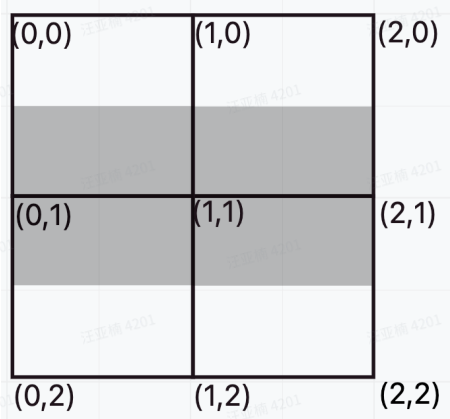


如果你仔细看你会发现画布中的1px宽线段是更加粗的，为什么呢？

思考：相信大家都遇过移动端1px线段变粗的场景，知道原因吗？这里属于同一种情况吗？

解答：

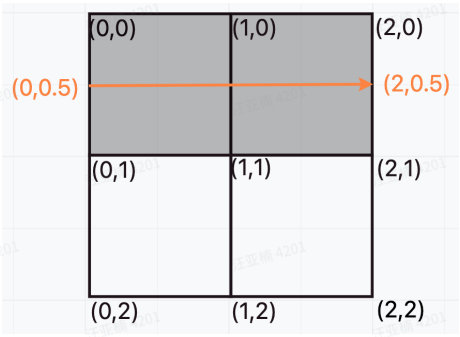
两种情况是不一样的，移动端线段变粗是多倍屏导致的，我们在css中设置1px，如果是二倍屏将用2px去展示所以看起来是变粗了，而这里不关多倍屏的事，而是浮点数引起的。



假如我们在画布中绘制 `(0,1)->(2,1)` 的1像素宽的线段， 线段实际是以路径为线段中心向两边各渲染 `0.5` 像素宽的线段，我们上面讲了浮点数像素是会引起 `铺平->柔化锯齿` 这个流程的，所以最后的效果看起来就是又粗又模糊。

那如何解决呢？

将线段向上或是向下偏移0.5像素，使得它的描边不会引起浮点数像素的产生，如下图：



（示例中第一根线条未做偏移，第二个做了0.5偏移）

https://codepen.io/huanjinliu/pen/bGQMzQE

5）如何对待浮点数：“合理取整，效果优先”策略

虽说浮点数可怕，但是浮点数的使用基本是无法规避的，所以咱们采用的是“合理取整，效果优先”的策略。以下情况可以考虑取整（写文章那刻能想到的，仅供参考~ω~）：

1. 粒子效果中位置坐标和尺寸的取整，未取整的图像效果将十分模糊。

未取整效果

取整后效果

（点击画布可切换是否取整，默认取整）

https://codepen.io/huanjinliu/pen/abQGXaG

2. 大尺寸画布中绘制大图。这种情况下图像浮点数的偏移或尺寸的近乎没有一点好处，但它的存在却会让边缘变得模糊。

6）浮点数造成的性能问题

上面也谈到了，浮点数像素的存在使得画布不得不额外多做一些工作，从计算机的角度看就是多做了很多额外的运算，所以理论上会引起性能问题，应该尽可能取整。

但是呢，这里提出一个不太一样的观点，浮点数造成的性能问题其实没那么严重，对于粒子效果，取整操作耗的性能也不少；而对于图像绘制而言，大图往往伴随缩放一起用，缩放基本就会产生浮点数像素，这很难避免，小图呢对性能的影响又微乎其微（如不缩放，让大图像偏移浮点数像素，实际测试效果也基本对性能没影响），所以我觉得大可不必单纯为了提高性能而对浮点数赶尽杀绝，反之我们应该更多关注它对图像效果的影响。

7）拒绝平滑，拒绝模糊

既然浏览器提供的“平滑服务”会导致边缘模糊，那问题来了，我们能拒绝这个服务吗？

可以的，但是仅限于使用 `drawImage()` 绘制图像的场景下，点击前往学习：应用4 · 图像绘制